Índices, nulos, expresiones y agregaciones

Buscar más rápido, tratar los datos que faltan, calcular y resumir

Semana 09 - Sesión 18

Rel Guzman



Logro de la sesión

Al finalizar, podrás explicar **qué es un índice**, tratar los **valores nulos**, calcular con **expresiones** y resumir datos con **agregaciones**.

Revisión de la sesión anterior: ¿qué hace un `JOIN`?

`cliente`

id	nombre
1	Ana
2	Bruno
3	Carla

tabla **principal**

`pedido`

cliente_id	producto
1	Laptop
1	Mouse
3	Teclado
4	Monitor

la **otra** tabla

Tienes una **tabla principal** y buscas **coincidencias** en otra tabla, usando una **columna en común**.

```
ON cliente.id = pedido.cliente_id
```

- **Ana** tiene dos coincidencias y **Carla** una.
- Bruno no tiene ninguna: no hizo pedidos.
- El Monitor tampoco: su `cliente_id` es 4 y ese cliente no existe.

La pregunta de siempre

Cada tipo de `JOIN` decide **qué hacer con las filas que no encontraron coincidencia**.

Revisión de la sesión anterior: los cuatro `JOIN`

	Tipo	Qué conserva	En el ejemplo
	`INNER JOIN`	solo las coincidencias	Ana y Carla. Bruno no aparece.
	`LEFT JOIN`	toda la tabla izquierda	Ana, Bruno y Carla. Bruno con *NULL*.
	`RIGHT JOIN`	toda la tabla derecha	todos los pedidos, también el Monitor.
	`FULL OUTER JOIN`	todo, de ambas tablas	todos; lo que no coincide va con *NULL*.

LEFT JOIN

«mi tabla principal **manda**»

INNER JOIN

«solo me interesan las **coincidencias**»

FULL JOIN

«quiero **todo**, coincida o no»

Revisión de la sesión anterior: leer un `LEFT JOIN`

```sql
SELECT c.id, c.nombre, p.producto
FROM cliente c
LEFT JOIN pedido p
   ON c.id = p.cliente_id;
```

resultado

id	nombre	producto
1	Ana	Laptop
1	Ana	Mouse
2	Bruno	*NULL*
3	Carla	Teclado

En palabras simples

«Mi tabla principal es `cliente`. Quiero conservar **todos** los clientes y buscar coincidencias en `pedido`. Si las encuentro, agrego esos datos. Si no, pongo *NULL*.»

Ojo con esto

Una fila de la tabla principal **se repite** si tiene varias coincidencias: Ana sale **dos veces** porque tiene dos pedidos.

Índices

Sin índice: buscar fila por fila

cliente

id	nombre
1	Rosa
2	Beto
3	Zoe
4	Ana
5	Luis

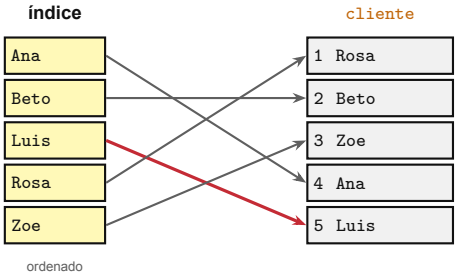
```
SELECT * FROM cliente
WHERE nombre = 'Luis';
```

- Los nombres están **desordenados**.
- Hay que preguntar **fila por fila**: ¿Rosa? no. ¿Beto? no. ¿Zoe? no…
- Con 5 filas no se nota. Con **un millón**, sí.

En palabras simples

Es buscar una palabra en un libro **leyéndolo completo**, página por página.

Con índice: ir directo a la fila



- Un índice es una **lista ordenada** de una columna.
- Cada nombre **señala** dónde está su fila.
- La tabla **queda igual**.

En palabras simples

Es el **índice de un libro**: buscas la palabra en la lista ordenada y te dice en qué página está.

`CREATE INDEX`: crear un índice

```
CREATE INDEX idx_cliente_nombre
ON cliente (nombre);
```

◄ el **nombre** que le pones al índice

◄ la **tabla** y la **columna**

- Se crea **una sola vez**.
- Tus consultas **se escriben igual**: solo salen más rápido.
- Sirve en las columnas por las que **buscas seguido**.

Ojo con esto

No se le pone índice a todo: cada índice **ocupa espacio** y hace un poco más lento **guardar** datos nuevos.

Demostración: buscar entre un millón de contactos

La carpeta `indices/` trae **la misma agenda dos veces**. Solo cambia **una instrucción**.

sin-indice

```
CREATE TABLE contacto (
   id      INTEGER PRIMARY KEY,
   nombre  TEXT,
   correo  TEXT
);
```

con-indice

```
-- la misma tabla, y además:
CREATE INDEX idx_contacto_correo
   ON contacto (correo);
```

	Sin índice	Con índice
Buscar un contacto por su correo tarda	_____ ms	_____ ms

Para probarlo, en cada carpeta: `npm install`, `npm run crear` y `npm run dev`.

Práctica 1 ¿Índice sí o no?

La tabla `alumno` de una universidad tiene **50 000 filas**.

Situación	¿Creo un índice? ¿Por qué?
Secretaría busca todos los días por `apellido`.	
La columna `observacion` casi nunca se consulta.	
Otra tabla, `dia_semana`, tiene solo 7 filas.	

Para escribir: la instrucción que crea el índice que sí hace falta.

Valores nulos

NULL: el dato que falta

empleado

id	nombre	area	sueldo	anio_ingreso	telefono
1	Ana	Ventas	2500	2018	987111222
2	Bruno	Ventas	1800	2021	*NULL*
3	Carla	Sistemas	4200	2015	955333444
4	Diego	Sistemas	3100	2023	''
5	Elena	Almacén	1500	2020	*NULL*

- *NULL* significa **«no se sabe»** o **«no hay dato»**.
- Bruno y Elena **no registraron** teléfono.
- Diego tiene '': un **texto vacío**. Parece lo mismo, pero **no es** *NULL*.

Valor	Qué dice
0	un número: «tiene cero»
''	un texto sin letras: «lo dejó en blanco»
NULL	**ningún valor**: «no lo sabemos»

En palabras simples

En un examen: un **cero** es una nota. Un casillero **sin nota** es otra cosa: todavía no se sabe cuánto sacó.

`IS NULL`: preguntar si falta el dato

Pregunta: ¿qué empleados **no tienen** teléfono registrado?

```sql
SELECT nombre, telefono
FROM empleado
WHERE telefono IS NULL;
```

resultado

nombre	telefono
Bruno	*NULL*
Elena	*NULL*

- Lo contrario es `IS NOT NULL`: los que **sí** tienen un valor.
- Diego **no sale**: su teléfono es `''`, y eso es un valor.

Ojo con esto

`WHERE telefono = NULL` **no devuelve nada**. Comparar con algo desconocido nunca da «verdadero». Para los nulos se usa siempre `IS NULL`.

COALESCE: un valor de repuesto

```
SELECT nombre,
   COALESCE(telefono, 'sin teléfono') AS contacto
FROM empleado;
```

resultado

nombre	contacto
Ana	987111222
Bruno	sin teléfono
Carla	955333444
Diego	''
Elena	sin teléfono

- `COALESCE(a, b)` devuelve `a`; y si `a` es *NULL*, devuelve `b`.
- Acepta más valores: `COALESCE(a, b, c)` devuelve **el primero que no sea** *NULL*.
- Solo cambia lo que **se muestra**: la tabla sigue guardando *NULL*.

En palabras simples

Es el **suplente**: si el titular no vino, entra el que sigue en la lista.

NULLIF: convertir un valor en NULL

```sql
SELECT nombre,
    NULLIF(telefono, '') AS telefono
FROM empleado;
```

resultado

nombre	telefono
Ana	987111222
Bruno	*NULL*
Carla	955333444
Diego	*NULL*
Elena	*NULL*

- `NULLIF(a, b)` devuelve *NULL* si `a` **es igual** a `b`. Si no, devuelve `a`.
- Juntos: `COALESCE(NULLIF(telefono, ''), 'sin teléfono')` y Diego también sale «sin teléfono».

Cómo se ve en la realidad

El uso más común es **evitar dividir entre cero**: `total / NULLIF(cantidad, 0)` da *NULL* en vez de un error.

Práctica 2 Valores nulos en la tienda

producto

id	nombre	categoria	precio	stock
1	Laptop	Cómputo	3200	5
2	Mouse	Cómputo	45	*NULL*
3	Silla	Muebles	380	12
4	Mesa	Muebles	650	0
5	Lámpara	Hogar	90	*NULL*

El `stock` en *NULL* quiere decir que **todavía no se contó**.

El `stock` en `0` quiere decir que **se contó y no hay**.

Pregunta	Respuesta
¿Cuántas filas devuelve `WHERE stock IS NULL`?	
¿Cuántas filas devuelve `WHERE stock = NULL`?	
¿Qué muestra `COALESCE(stock, 0)` para el Mouse?	
¿Qué muestra `NULLIF(stock, 0)` para la Mesa?	

Para escribir: la consulta que muestra el nombre y el stock, con el texto `'por contar'` donde falte.

Expresiones

Una expresión: calcular dentro del `SELECT`

```
SELECT nombre,
    sueldo * 12 AS sueldo_anual
FROM empleado;
```

- una columna **tal cual** está en la tabla
- una **expresión**: un cálculo, con su nombre
- la tabla **no cambia**

resultado

nombre	sueldo_anual
Ana	30000
Bruno	21600
Carla	50400
Diego	37200
Elena	18000

- Una **expresión** combina columnas, valores y operadores: `+ - * /`
- El cálculo se hace **en cada fila**.
- `AS` le pone nombre a la columna nueva.
- La columna `sueldo_anual` **no se guarda**: se calcula al consultar.

Expresiones con fechas: la antigüedad

Pregunta: ¿cuántos años lleva cada empleado en la empresa?

```sql
SELECT nombre,
   CAST(strftime('%Y', 'now') AS INTEGER) - anio_ingreso
      AS antiguedad
FROM empleado;
```

resultado

nombre	antiguedad
Ana	8
Bruno	5
Carla	11
Diego	3
Elena	6

Pieza	Qué hace
`strftime('%Y', 'now')`	el año de hoy, como **texto**: `'2026'`
`CAST(... AS INTEGER)`	lo convierte en **número**: `2026`
`- anio_ingreso`	le resta el año en que entró cada uno

Así se escribe en **SQLite**. En MySQL la misma idea es `YEAR(CURDATE()) - anio_ingreso`.

Concatenar: pegar textos con ||

```sql
SELECT nombre || ' (' || area || ')' AS info
FROM empleado;
```

resultado

info
Ana (Ventas)
Bruno (Ventas)
Carla (Sistemas)
Diego (Sistemas)
Elena (Almacén)

- `||` **pega** un texto a continuación del otro.
- Lo que va entre comillas se escribe **tal cual**: aquí, los paréntesis y el espacio.
- También pega números: `nombre || ' - ' || sueldo` da `Ana - 2500`.

Ojo con esto

Si **una** de las piezas es *NULL*, **todo** el resultado es *NULL*. Se arregla con `COALESCE(telefono, 'sin teléfono')`.

En MySQL se usa `CONCAT(nombre, ' (', area, ')')`.

`CASE WHEN`: decidir fila por fila

```sql
SELECT nombre, sueldo,
   CASE
      WHEN sueldo >= 3000 THEN 'alto'
      WHEN sueldo >= 2000 THEN 'medio'
      ELSE 'bajo'
   END AS nivel
FROM empleado;
```

resultado

nombre	sueldo	nivel
Ana	2500	medio
Bruno	1800	bajo
Carla	4200	alto
Diego	3100	alto
Elena	1500	bajo

- Las condiciones se revisan **en orden**: se queda con **la primera** que se cumple.
- `ELSE` atrapa a todos los demás. Sin `ELSE`, a esos les toca *NULL*.
- Siempre se cierra con `END`.

En palabras simples

Es la regla del profesor: «de 18 para arriba, sobresaliente; de 11 para arriba, aprobado; **si no**, desaprobado».

Funciones de texto

Una **función** recibe un valor y devuelve otro. Estas trabajan con **texto**.

Función	Qué hace	Con `nombre = 'Carla'`	Resultado
`UPPER`	pasa todo a mayúsculas	`UPPER(nombre)`	`CARLA`
`LOWER`	pasa todo a minúsculas	`LOWER(nombre)`	`carla`
`LENGTH`	cuenta los caracteres	`LENGTH(nombre)`	`5`
`SUBSTR`	saca un pedazo: desde dónde y cuántos	`SUBSTR(nombre, 1, 3)`	`Car`
`REPLACE`	cambia un texto por otro	`REPLACE(nombre, 'a', 'o')`	`Corlo`

```sql
SELECT UPPER(nombre) AS nombre, SUBSTR(area, 1, 3) AS sigla   -- CARLA | Sis
FROM empleado;
```

En `SUBSTR` las posiciones empiezan en **1**, no en 0.

Funciones numéricas

```sql
SELECT nombre,
   ROUND(sueldo / 30.0, 2) AS sueldo_diario,
   ABS(sueldo - 2620) AS distancia
FROM empleado;
```

resultado

nombre	sueldo_diario	distancia
Ana	83.33	120
Bruno	60.0	820
Carla	140.0	1580
Diego	103.33	480
Elena	50.0	1120

Función	Qué hace	Ejemplo	Resultado
`ROUND`	redondea a los decimales que pidas	`ROUND(83.3333, 2)`	`83.33`
`ABS`	quita el signo: el valor absoluto	`ABS(-820)`	`820`

`2620` es el sueldo promedio: `distancia` dice **cuánto se aleja** cada uno, por arriba o por abajo.

Ojo con esto

Entre dos enteros, `/` **bota los decimales**: `2500 / 30` da `83`. Por eso se divide entre `30.0`.

Expresiones en la tienda

producto

id	nombre	categoria	precio	stock
1	Laptop	Cómputo	3200	5
2	Mouse	Cómputo	45	*NULL*
3	Silla	Muebles	380	12
4	Mesa	Muebles	650	0
5	Lámpara	Hogar	90	*NULL*

```sql
SELECT UPPER(nombre) AS nombre,
   ROUND(precio * 1.18, 1) AS precio_final,
   CASE
      WHEN precio >= 500 THEN 'caro'
      ELSE 'económico'
   END AS tipo
FROM producto;
```

Pregunta	Respuesta
¿Qué sale en `nombre` para la fila 3?	
¿Cuánto es el `precio_final` del Mouse?	
¿Cuántos productos salen como `'caro'`?	

Para escribir: la consulta que muestra `Laptop [Cómputo]`, `Mouse [Cómputo]`… en una columna llamada `etiqueta`.

Agregaciones

Agregaciones: muchas filas, un solo valor

empleado

id	nombre	area	sueldo	anio_ingreso	telefono
1	Ana	Ventas	2500	2018	987111222
2	Bruno	Ventas	1800	2021	*NULL*
3	Carla	Sistemas	4200	2015	955333444
4	Diego	Sistemas	3100	2023	''
5	Elena	Almacén	1500	2020	*NULL*

- Una expresión calcula **fila por fila**.
- Una **agregación** toma **toda una columna** y devuelve **un solo valor**.

Función	Qué responde	Ejemplo	Resultado
`COUNT`	¿cuántas filas hay?	`COUNT(*)`	5
`SUM`	¿cuánto suman?	`SUM(sueldo)`	13100
`AVG`	¿cuál es el promedio?	`AVG(sueldo)`	2620.0
`MIN`	¿cuál es el menor?	`MIN(sueldo)`	1500
`MAX`	¿cuál es el mayor?	`MAX(sueldo)`	4200

```sql
SELECT COUNT(*) AS cantidad, AVG(sueldo) AS sueldo_promedio FROM empleado;
```

Las agregaciones y los `NULL`

```sql
SELECT COUNT(*) AS empleados,
   COUNT(telefono) AS con_telefono
FROM empleado;
```

resultado

empleados	con_telefono
5	3

Se escribe	Cuenta
`COUNT(*)`	**todas** las filas
`COUNT(telefono)`	solo las filas donde `telefono` **no es** *NULL*

- Bruno y Elena no se cuentan. Diego **sí**: su `''` es un valor.
- `SUM`, `AVG`, `MIN` y `MAX` también **saltan** los *NULL*: no los toman como cero.

`GROUP BY`: un resultado por grupo

Pregunta: ¿cuántos empleados hay y cuál es el sueldo promedio **en cada área**?

```
SELECT area,
   COUNT(*) AS cantidad,
   AVG(sueldo) AS sueldo_promedio
FROM empleado
GROUP BY area;
```

resultado

area	cantidad	sueldo_promedio
Almacén	1	1500.0
Sistemas	2	3650.0
Ventas	2	2150.0

1. Se **juntan** las filas que tienen la misma `area`: salen tres grupos.
2. La agregación se calcula **dentro de cada grupo**.
3. El resultado tiene **una fila por grupo**.

Ojo con esto

En el `SELECT` solo van las columnas del `GROUP BY` y las agregaciones. Pedir también `nombre` no tiene sentido: ¿el de cuál de los dos de Ventas?

HAVING: filtrar los grupos

Pregunta: ¿qué áreas tienen **dos o más** empleados?

```sql
SELECT area, COUNT(*) AS cantidad
FROM empleado
GROUP BY area
HAVING COUNT(*) >= 2;
```

resultado

area	cantidad
Sistemas	2
Ventas	2

	Qué filtra	Cuándo
`WHERE`	**filas** sueltas	**antes** de agrupar
`HAVING`	**grupos** completos	**después** de agrupar

Ojo con esto

`WHERE COUNT(*) >= 2` da error: cuando actúa el `WHERE`, los grupos **todavía no existen**.

Todo junto: el orden de una consulta

```
SELECT area, AVG(sueldo) AS sueldo_promedio
FROM empleado
WHERE anio_ingreso >= 2018
GROUP BY area
HAVING AVG(sueldo) > 2000
ORDER BY sueldo_promedio DESC;
```

- ◀ qué se muestra de cada grupo
- ◀ de qué tabla
- ◀ qué **filas** entran
- ◀ cómo se **agrupan**
- ◀ qué **grupos** se quedan
- ◀ en qué orden salen

resultado

area	sueldo_promedio
Sistemas	3100.0
Ventas	2150.0

- El `WHERE` deja fuera a **Carla**, que entró en 2015.
- Quedan tres grupos: Ventas, Sistemas y Almacén.
- El `HAVING` deja fuera a **Almacén**: su promedio es 1500.

Las cláusulas se escriben **siempre en este orden**.

Práctica 4 Haz tú el `GROUP BY`, a mano

producto

id	nombre	categoria	precio	stock
1	Laptop	Cómputo	3200	5
2	Mouse	Cómputo	45	*NULL*
3	Silla	Muebles	380	12
4	Mesa	Muebles	650	0
5	Lámpara	Hogar	90	*NULL*

```sql
SELECT categoria,
   COUNT(*) AS cantidad,
   MAX(precio) AS precio_mayor
FROM producto
GROUP BY categoria;
```

1. **Agrupa:** en la tabla, encierra juntas las filas de la misma `categoria`.
2. **Completa** el resultado: una fila por cada grupo.
3. **Tacha** la fila que desaparece si agregamos `HAVING COUNT(*) >= 2`.

resultado

categoria	cantidad	precio_mayor

Trabaja **en pareja**: uno agrupa, el otro completa. Luego comparen con la pareja de al lado.

Preguntas



Cierre: lo de hoy en una tabla

	En una frase
Índice	Una lista ordenada que dice dónde está cada fila.
`IS NULL`	Pregunta si falta el dato. Nunca `= NULL`.
`COALESCE` / `NULLIF`	Poner un repuesto al *NULL*; convertir un valor en *NULL*.
Expresión	Un cálculo en el `SELECT`, fila por fila, con su `AS`.
`CASE WHEN`	Decide un valor según una condición.
Agregación	Muchas filas, un solo valor: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.
`GROUP BY` / `HAVING`	Un resultado por grupo; quedarse solo con algunos grupos.

Para no confundirlos: `WHERE` filtra **filas**, `HAVING` filtra **grupos**.

Referencias

Coronel, C., Morris, S., & Rob, P. (2011). *Bases de datos: diseño, implementación y administración* (9.ª ed.). Cengage Learning.

Elmasri, R., & Navathe, S. (2007). *Fundamentos de sistemas de bases de datos* (5.ª ed.). Addison Wesley.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill.

SQLite. (s. f.). *Query planning*. `https://www.sqlite.org/queryplanner.html`

SQLite. (s. f.). *Built-in scalar SQL functions*. `https://www.sqlite.org/lang_corefunc.html`

SQLite. (s. f.). *Built-in aggregate functions*. `https://www.sqlite.org/lang_aggfunc.html`



UTP Universidad Tecnológica del Perú